GNNExplainer: Generating Explanations for Graph Neural Networks

Ying et al.

Arion Scheid

TU Wien

May 2026

Motivation

- GNNs:
  - Powerful for learning graph structured data
  - Incorporate graph structure **and** feature information $\Rightarrow$ Complex models
  - Lack of interpretability
- Goal: Generate explanations!
  - Increases trust in model
  - Improves transparency (for fairness, privacy, safety)
  - Helps identify network characteristics, systematic patterns of mistakes
- Problem: GNNs are complex, so explanations are hard!

Explanation Idea

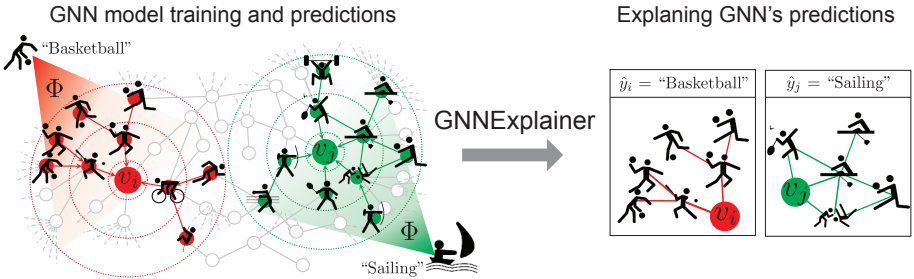


Figure: Explaining hobby predictions with social network graph[1]

[1] Ying et al., GNNExplainer: Generating Explanations for Graph Neural Networks, 2019

Computation graph

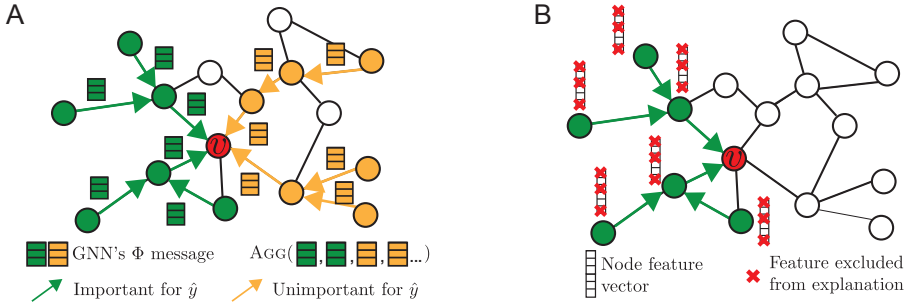


Figure: Explanations consist of identifying relevant nodes and features [2]

[2] Ying et al., GNNExplainer: Generating Explanations for Graph Neural Networks, 2019

Notation and setup

Throughout this presentation we use the following notation:

- $G = (V, E)$ — a graph with node set V and edge set E
- $X \in \mathbb{R}^{|V| \times d}$ — node feature matrix (d features per node)
- Φ — a **trained, frozen** GNN (weights are fixed during explanation)
- $\hat{y} = \Phi(G, X)_v$ — the GNN's predicted label for node v
- $G_c(v)$ — the **computation graph** of node v: the L-hop neighbourhood that the L-layer GNN aggregates over to produce $\hat{y}$
- A_c — the adjacency matrix of $G_c(v)$
- Y — the predicted label viewed as a **random variable** (captures the GNN's softmax distribution over classes $1, \ldots, C$)

Goal: Find a small subgraph $G_S \subseteq G_c$ and a feature subset $X_S \subseteq X$ that are most important for the prediction $\hat{y}$.

Entropy — measuring uncertainty

Shannon entropy measures the uncertainty of a discrete random variable Y with possible outcomes $\{1, \ldots, C\}$:

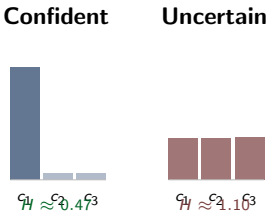
$$H(Y) \;=\; -\sum_{c=1}^{C} P(Y = c)\, \log P(Y = c). \tag{1}$$

Properties:

- $H(Y) \geq 0$; equals 0 only when one outcome has probability 1 (no uncertainty).
- Maximized when Y is uniform, i.e. $P(Y = c) = 1/C$ for all c.

Example ($C = 3$ classes):

- Confident: $P = (0.9, 0.05, 0.05) \Rightarrow H \approx 0.47$
- Uncertain: $P = (0.33, 0.33, 0.34) \Rightarrow H \approx 1.10$



Conditional entropy and mutual information

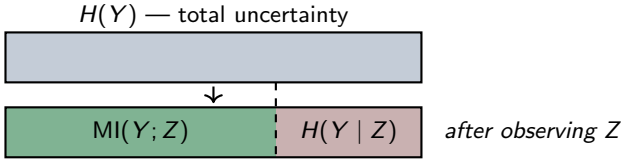
Conditional entropy — residual uncertainty in Y *after* observing evidence Z:

$$H(Y \mid Z) \;=\; -\sum_{c=1}^{C} P(Y=c \mid Z)\,\log P(Y=c \mid Z). \tag{2}$$

Conditioning on informative evidence reduces entropy: $H(Y \mid Z) \leq H(Y)$.

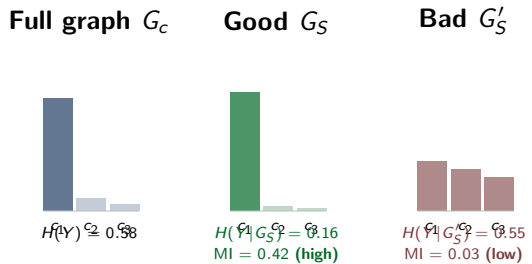
Mutual information — how much observing Z reduces our uncertainty about Y:

$$\mathrm{MI}(Y;Z) \;=\; H(Y) \;-\; H(Y \mid Z) \;\geq\; 0. \tag{3}$$



Key insight: A good explanation $Z = (G_S, X_S)$ should have *high* MI — observing it should make the prediction nearly certain (low $H(Y \mid Z)$).

Intuition: how subgraph choice affects the prediction



- **Good subgraph** G_S: contains the relevant structure $\Rightarrow$ GNN remains confident $\Rightarrow$ low $H(Y \mid G_S)$, high MI.
- **Bad subgraph** G'_S: misses key edges $\Rightarrow$ GNN becomes uncertain $\Rightarrow$ high $H(Y \mid G'_S)$, low MI.

Formal objective: maximize mutual information

For node v with computation graph G_c, find the subgraph $G_S \subseteq G_c$ and features X_S that maximize MI with the prediction:

$$\max_{G_S, X_S} \mathrm{MI}(Y,\ (G_S, X_S)) \ = \ \underbrace{H(Y)}_{\text{constant}} \ - H(Y \mid G = G_S,\ X = X_S). \tag{4}$$

Since Φ is frozen, $H(Y)$ is fixed. Maximizing MI $\equiv$ minimizing conditional entropy:

$$\min_{G_S, X_S} \ H(Y \mid G = G_S,\ X = X_S) \ = \ -\sum_{c=1}^{C} P_\Phi(Y = c \mid G_S, X_S)\ \log P_\Phi(Y = c \mid G_S, X_S). \tag{5}$$

This treats Y as a full random variable: the loss uses the **entire softmax distribution** over all C classes. It penalizes any residual uncertainty — the GNN should be *confident*, regardless of which class it favours.

Class-specific formulation

In practice, we know the GNN's predicted label $\hat{y} = c^*$. We can replace the full entropy with a **cross-entropy targeted at** c^*:

$$\min_{G_S, X_S} \; -\sum_{c=1}^{C} \mathbb{1}[\hat{y} = c] \; \log P_\Phi(Y = c \mid G_S, X_S) \; = \; -\log P_\Phi(Y = c^* \mid G_S, X_S). \tag{6}$$

Conditional entropy (Eq. 5):

- Uses full distribution over all C classes
- Asks: "Is the GNN confident?"
- Penalizes *any* uncertainty

Class-specific (Eq. 6):

- Only looks at $P(Y = c^*)$
- Asks: "Does G_S support predicting c^*?"
- Used in practice by Ying et al.

Both formulations agree when the GNN is highly confident. The class-specific form is simpler and directly interpretable as the standard **cross-entropy loss** used in classification.

Continuous relaxation with learnable masks

Searching over all subsets $G_S \subseteq G_c$ is combinatorial ($2^{|E|}$ options). Instead, learn continuous masks:

- **Edge mask** $M \in \mathbb{R}^{|E|}$: one scalar per edge, passed through sigmoid $\sigma(\cdot)$ to obtain weights in $[0, 1]$. The masked adjacency is $A_c \odot \sigma(M)$, where $\odot$ is element-wise multiplication.
- **Feature mask** $F \in [0, 1]^d$: one scalar per feature dimension. The masked features are $X_S = X_c \odot F$ (features not selected are zeroed out).

Substituting into the class-specific objective:

$$\min_{M,\,F} \; -\log P_\Phi(Y = \hat{y} \mid G = A_c \odot \sigma(M),\; X = X_c \odot F)\,. \tag{7}$$

Both M and F are optimized jointly via gradient descent through the frozen GNN Φ.

Regularization

The raw objective (Eq. 7) can produce dense, uninterpretable masks. Regularization terms encourage **sparse** and **discrete** explanations:

$$\mathcal{L} = \underbrace{-\log P_\Phi(Y = \hat{y} \mid A_c \odot \sigma(M),\, X_c \odot F)}_{\text{cross-entropy (fidelity)}} + \lambda_1 \underbrace{\|\sigma(M)\|_1}_{\text{size}} + \lambda_2 \underbrace{\sum_i H_{\text{elt}}(\sigma(M_i))}_{\text{element-wise entropy}} \tag{8}$$

where $H_{\text{elt}}(p) = -p \log p - (1-p)\log(1-p)$ is the **binary entropy** of a single mask element.

- **Size regularization** ($\|\sigma(M)\|_1$): penalizes the total "mass" of the mask, encouraging few edges to be selected $\Rightarrow$ compact explanation.
- **Element-wise entropy** (H_{elt}): pushes each $\sigma(M_i)$ toward 0 or 1 (since H_{elt} is maximized at 0.5 and zero at 0 or 1). This discourages ambiguous, half-on mask values $\Rightarrow$ crisp, interpretable masks.

Perturbation-based explainers

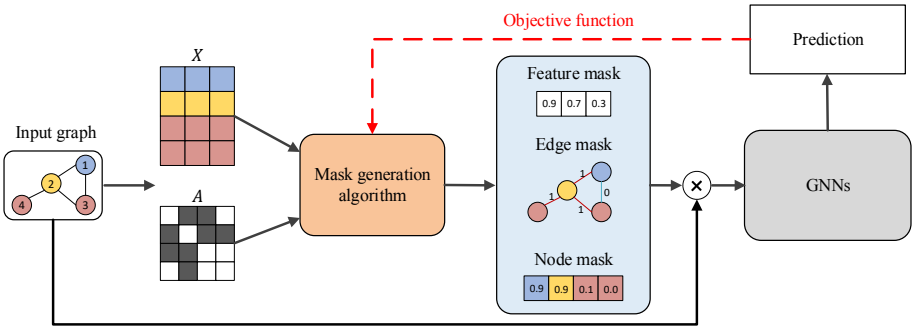


Figure: General pipeline of perturbation-based explainers[3]

[3] Yuan et al., Explainability in Graph Neural Networks: A Taxonomic Survey, 2023

Research Questions

- Are the explanations sensible?
  - Can they identify ground truths?
  - Can they make sense of real-world data?
- How do other (newer) models compare?
- **Structural limitation:** Can gradient-based mask optimization get stuck?

Evaluation setup

For each target node, the explainer optimizes $\mathcal{L}$ (Eq. 8), producing edge weights $\sigma(M_i) \in [0, 1]$. We threshold at 0.5: edges with $\sigma(M_i) > 0.5$ are **selected**; the rest are discarded.

Accuracy measures overall GNN classification quality (before explaining):

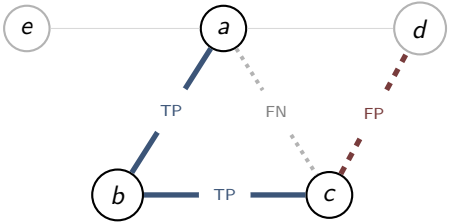
$$\text{Acc} = \frac{\text{number of correctly classified nodes}}{\text{total nodes}}. \tag{9}$$

To evaluate *explanation* quality, we compare selected edges against ground-truth edges using **precision**, **recall**, and **F1 score**:

$$P = \frac{|\text{selected} \cap \text{ground truth}|}{|\text{selected}|}, \quad R = \frac{|\text{selected} \cap \text{ground truth}|}{|\text{ground truth}|}, \quad F_1 = \frac{2PR}{P+R}. \tag{10}$$

- P — of the edges the explainer highlights, how many are truly relevant?
- R — of the truly relevant edges, how many does the explainer find?
- F_1 — harmonic mean; balances both (high only when both P and R are high).

F1 example: triangle in a small graph



Ground truth: triangle $\{a, b, c\}$ (3 edges).

The explainer selects edges $\{a\text{-}b,\ b\text{-}c,\ c\text{-}d\}$.

TP (correct)	a-b, b-c	$\Rightarrow 2$
FP (spurious)	c-d	$\Rightarrow 1$
FN (missed)	a-c	$\Rightarrow 1$

$$P = \frac{2}{2+1} = 0.67$$

$$R = \frac{2}{2+1} = 0.67$$

$$F_1 = \frac{2 \times 0.67 \times 0.67}{0.67 + 0.67} = 0.67$$

A perfect explanation would recover all 3 triangle edges and nothing else $\Rightarrow F_1 = 1.0$.

Synthetic examples

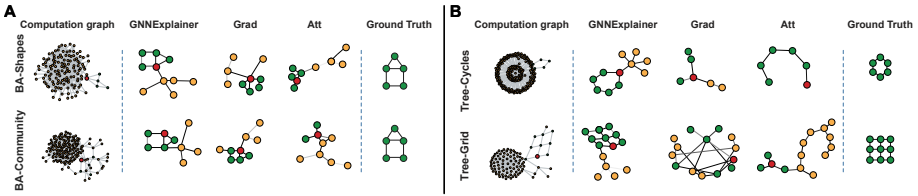


Figure: Examples with known ground truths for evaluation[4]

[4] Ying et al., GNNExplainer: Generating Explanations for Graph Neural Networks, 2019

Real-world examples

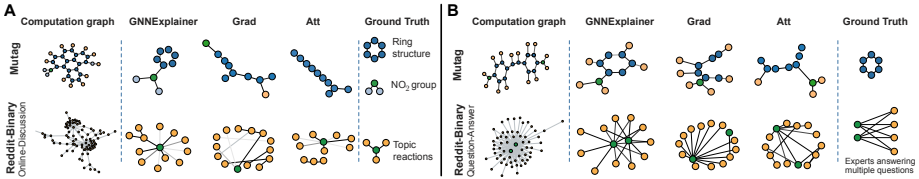


Figure: Real-world examples for evaluation[5]

[5] Ying et al., GNNExplainer: Generating Explanations for Graph Neural Networks, 2019

Local optima failure mode

- Gradient descent finds the *nearest* local minimum, not the global optimum
- Problem arises when sub-structures of the correct explanation are *also* locally sufficient (i.e., already maintain high prediction confidence)
- Example: node in a k-clique embedded in sparse Erdős-Rényi graph
  - Erdős-Rényi: random graph where each edge exists independently with probability p
  - A triangle within the k-clique is already denser than the sparse background
  - Optimizer stops at triangle — a *sub-clique local minimum*
- Number of competing sub-optima grows **exponentially** with clique size k

Multi-size clique experiment

- Novel benchmark: cliques of size 3, 4, 5 embedded in sparse Erdős-Rényi graph ($n = 200$, $p = 0.02$)
- Binary labels (clique / non-clique); node features uniform $\Rightarrow$ GNN must rely on structure
- Results confirm local-optima failure:

Clique size	F1	Avg. expl. clique size
$k = 3$ (triangles)	0.256	1.71 / 3
$k = 4$	0.105	1.33 / 4
$k = 5$	0.006	1.07 / 5

F1 degrades sharply; optimizer almost always stops at a single edge (2-clique) for large cliques.